Departamento de Tecnología Digital

Tecsup TECNOLOGÍA CON SENTIDO



LABORATORIO N° 09

Desarrollo de Software

DOCENTE:
Bestard Aroche, Yunior

CURSO:

Desarrollo de Aplicaciones Empresariales
4 - C24 - Sección CD

DESARROLLO DE APLICACIONES EMPRESARIALES:

Capacidades

1. **Identificar la funcionalidad de React y de su entorno de desarrollo (Node.js, npm y Vite) dentro de una aplicación empresarial.**
2. **Configurar y ejecutar un proyecto de React con Vite e interpretar su estructura (index.html, main.jsx y App.jsx).**
3. **Construir una interfaz con JSX aplicando expresiones, listas con map y key, renderizado condicional, listas anidadas y estilos.**

I. Seguridad

- En este laboratorio está prohibida la manipulación del hardware, conexiones eléctricas o de red. Así como la ingesta de alimentos y bebidas.
- Ubicar maletines y/o mochilas en lugar destinado para tal fin.
- Dejar la mesa de trabajo y la silla utilizada limpias y ordenadas.

II. Fundamento teórico

- Revise el material de la Semana 9 (Introducción a React: qué es React y una SPA, Node.js, npm y Vite, estructura del proyecto, JSX, listas y renderizado condicional) antes del desarrollo del laboratorio.

III. Normas empleadas

- No aplica

IV. Recursos

- En este laboratorio, cada estudiante trabajará con una computadora con Windows 11.
- Los softwares requeridos ya se encuentran instalados en los laboratorios.
- Se requiere Node.js (versión LTS, que incluye npm), un editor de código (Visual Studio Code), un navegador web y una cuenta de GitHub para el control de versiones.
- La aplicación desarrollada desde la Semana 3 hasta la Semana 8 (incluida la API REST de la Semana 8) y, cuando corresponda, la investigación propia de siete entidades.

V. Metodología para el desarrollo de la tarea

- El desarrollo del laboratorio es individual

VI. Modo de entrega

- El laboratorio se entrega mediante este mismo documento Word, completando cada ejercicio con su respuesta y la justificación solicitada.
- Cada ejercicio que lo requiera debe incluir captura de pantalla del código (App.jsx y App.css), de la página en el navegador y de la terminal cuando se ejecuten comandos.
- El documento debe incluir la dirección (URL) del repositorio de GitHub donde se aloja el proyecto actualizado.
- El documento final se sube por el medio indicado por el docente (aula virtual / plataforma del curso).

VII. Procedimiento

Hasta la Semana 8, cada estudiante construyó en Django una aplicación con CRUD, relaciones, Admin, Templates, consultas avanzadas y una API REST que entrega sus datos en formato JSON. En esta sesión comienza el bloque del front-end: se crea un proyecto de React con Vite, dentro de una carpeta frontend en el mismo repositorio del proyecto Django. Todavía no se consume la API (eso se hace en la Semana 12): los datos se escriben a mano en arreglos con la misma forma que la respuesta de la API de la Semana 8.

El laboratorio se desarrolla en dos partes. En la primera, cada estudiante prepara el entorno, crea el proyecto y construye una interfaz para la aplicación de las Semanas 4 a 8. En la segunda parte,

deberá hacer lo mismo sobre su investigación propia (las siete entidades registradas en el Admin en la Semana 5), cumpliendo los criterios detallados en la sección correspondiente.

El objetivo es que el estudiante comprenda que React no reemplaza a Django: lo complementa. Lo que en un Template se escribía con {{ }}, {% for %} y {% if %}, en React se escribe con llaves, map() y condicionales de JavaScript. En clase la teoría se demuestra con una aplicación de cafetería; cada estudiante la traslada a su propia aplicación con ayuda de la siguiente tabla de equivalencias.

Concepto trabajado en clase	Equivalente en su aplicación
Lista de productos (en clase: productos de la cafetería, con id, nombre, precio y stock)	Un arreglo con al menos cinco registros de su entidad principal, con id, un nombre o título, un campo numérico y un campo de estado o cantidad.
Dato que activa un mensaje (en clase: stock igual a 0 muestra Agotado)	Un campo de su entidad que permita mostrar un mensaje distinto (estado, cantidad o disponibilidad).
Relación 1:N anidada (en clase: pedidos con sus detalles)	Su relación 1:N, 1:1 o su modelo intermedio, representada como un arreglo dentro de otro arreglo.
Forma de los datos (en clase: la respuesta de GET /api/productos/)	La respuesta de la API que creó en la Semana 8 para su entidad.

Navegador (http://localhost:5173/)

→ index.html (una sola página)

→ main.jsx (monta React)

→ App.jsx (componente principal)

→ JSX (llaves, map y condicionales)

→ Datos escritos a mano (arreglos)

→ HTML dibujado por React

→ Pantalla

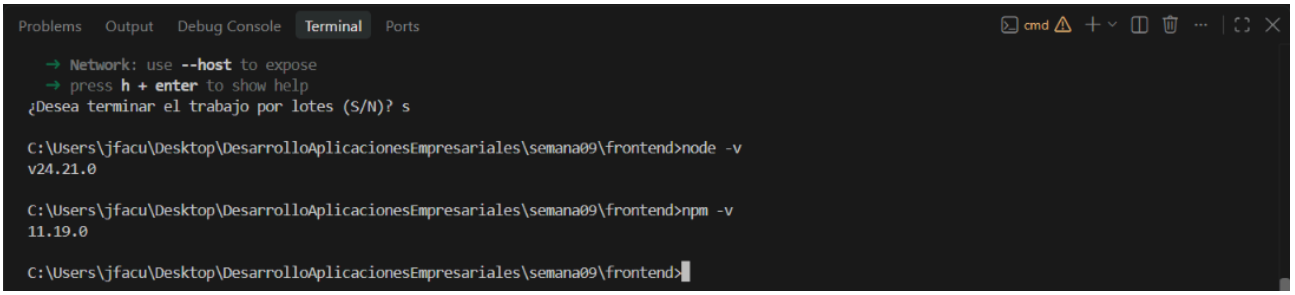
Al finalizar el laboratorio, el estudiante deberá ser capaz de instalar y ejecutar un proyecto de React con Vite, reconocer su estructura y construir una interfaz con JSX que muestre listas, mensajes condicionales, relaciones anidadas y estilos, y publicarla en GitHub.

Enunciados

PARTE 1 — Una interfaz React para la aplicación de las Semanas 4 a 8

Ejercicio 1 — Verificar el entorno: Node.js y npm

Verifique que Node.js y npm están instalados ejecutando node -v y npm -v en una terminal; si no lo están, instale la versión LTS desde nodejs.org. Documente las versiones obtenidas con una captura. Complete una tabla de equivalencias entre Python y Django, y Node.js y React, para estos elementos: el lenguaje, el gestor de paquetes, el archivo de dependencias, la carpeta de dependencias y el comando que levanta el servidor de desarrollo.



Se verificó la instalación de Node.js y npm en el entorno de desarrollo.

Elemento	Entorno Python / Django	Entorno Node.js / React
Lenguaje	Python	JavaScript / JSX
Gestor de Paquetes	pip	npm
Archivo de Dependencias	requirements.txt	package.json
Carpeta de Dependencias	venv/ (Entorno virtual)	node_modules/
Comando Servidor Dev	python manage.py runserver	npm run dev

Ejercicio 2 — Crear y ejecutar el proyecto con Vite

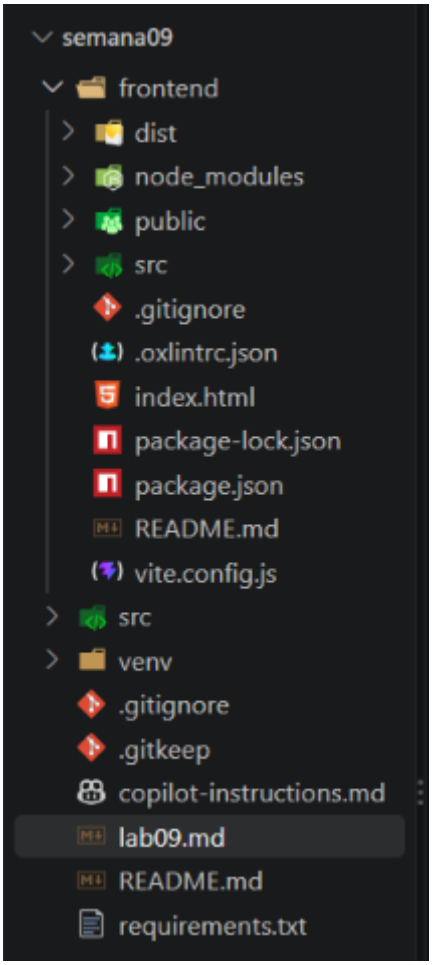
Dentro del repositorio de su proyecto Django, cree el proyecto de React con npm create vite@latest frontend -- --template react (si aparece un asistente, elija React y JavaScript). Ejecute npm install y npm run dev, abra http://localhost:5173/ y documente con capturas la salida de la terminal y la página inicial que muestra el navegador.



Se creó la estructura del proyecto Front-end dentro de la carpeta frontend/ mediante el comando npm create vite@latest. Se instalaron las dependencias base y se inició el servidor de desarrollo Vite en el puerto 5173.

Ejercicio 3 — Reconocer la estructura del proyecto

Explore la carpeta frontend y explique con sus palabras el papel de index.html, main.jsx, App.jsx, package.json y node_modules, relacionando cada uno con un elemento que ya conoce de Django (por ejemplo, requirements.txt o manage.py). Documente con una captura la estructura de carpetas.

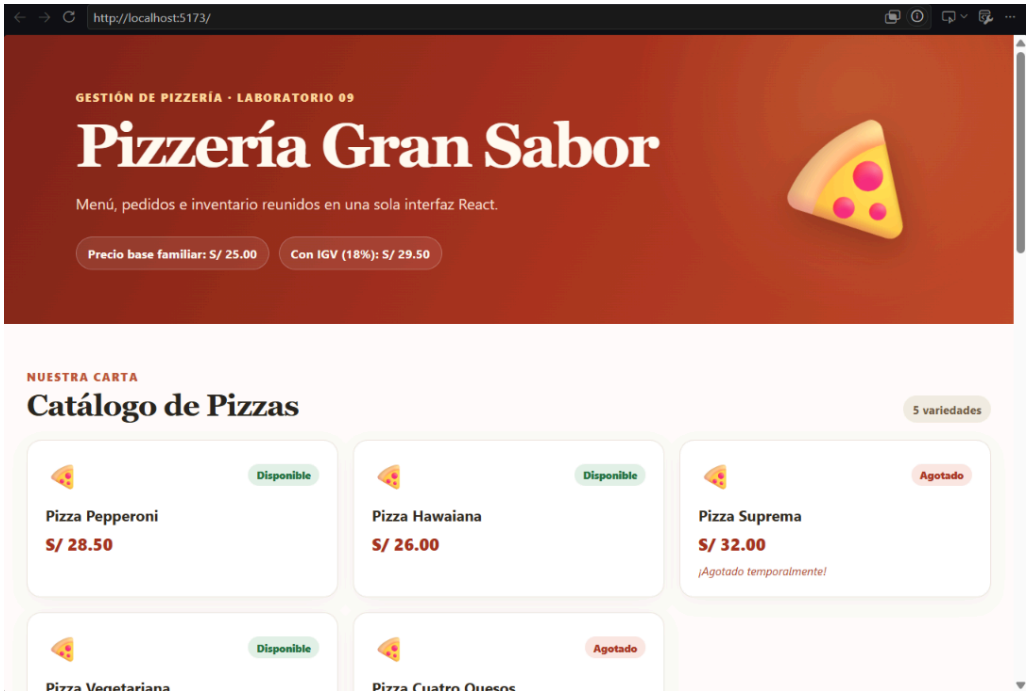


Equivalencias de la estructura:

- **index.html**: Punto de entrada HTML único de la Single Page Application (SPA), equivalente al maquetado principal base.html de Django.
- **src/main.jsx**: Archivo JavaScript que renderiza y monta el componente principal de React en el DOM HTML.
- **src/App.jsx**: Componente principal equivalente a las Vistas/Templates de Django.
- **package.json**: Gestor de manifiesto y scripts, equivalente a requirements.txt.
- **node_modules/**: Directorio de librerías instaladas, equivalente a venv/.

Ejercicio 4 — Escribir el primer JSX con expresiones

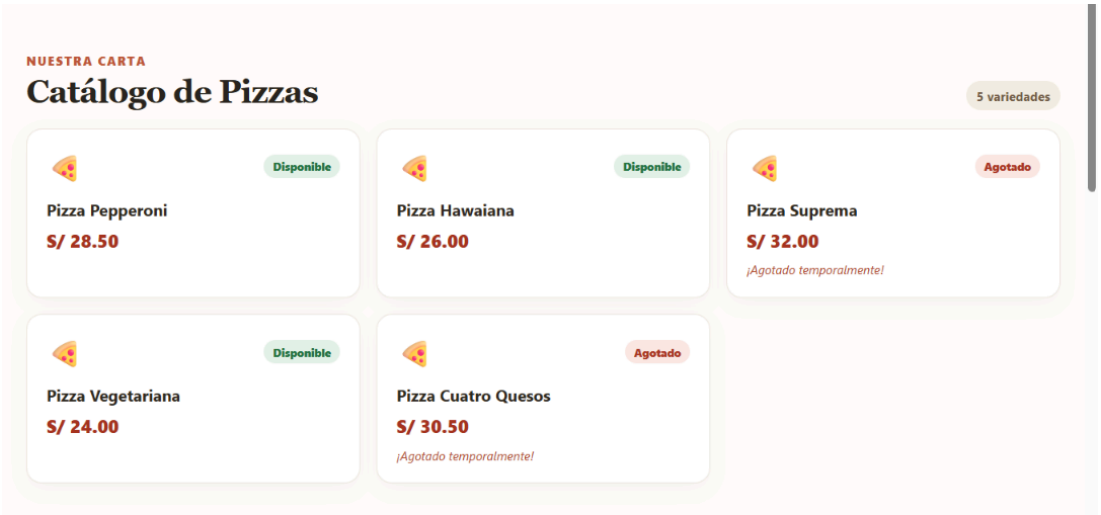
Vacíe el contenido de App.css y de index.css, y reemplace el contenido de App.jsx por un componente que muestre el nombre de su aplicación en un <h1>. Use llaves { } para insertar al menos dos expresiones: una variable y un cálculo o una llamada a una función (por ejemplo, un precio con impuesto usando toFixed(2)). Documente el código y la página en el navegador.



Se vaciaron los estilos por defecto y se utilizó JSX para insertar variables y cálculos en tiempo de ejecución entre llaves {}. Se aplicó el método .toFixed(2) para formatear el valor del precio base (precioBase) con su respectivo cálculo de impuesto (precioBase * 1.18).

Ejercicio 5 — Mostrar una lista con map() y key

Cree un arreglo con al menos cinco registros de su entidad principal, con la misma forma que la respuesta de la API de la Semana 8 (los decimales como texto, por ejemplo precio: "6.50"). Muéstrelos en una lista con map() y una key basada en el id, convirtiendo el decimal con Number(...).toFixed(2). Explique con sus palabras qué elemento de los Templates de Django equivale a este map().



Se definió el arreglo pizzas imitando la respuesta de la API REST de la Semana 08. Se renderizaron los elementos usando el método iterativo .map() asignando la propiedad key={pizza.id} a cada tarjeta.

El método .map() de JavaScript/JSX equivale al bloque {% for item in lista %} de los Templates de Django.

Ejercicio 6 — Aplicar renderizado condicional

Agregue a la lista un mensaje distinto según un campo de su registro, usando un ternario (por ejemplo, Agotado o Stock: n) y otro mensaje con && (por ejemplo, ¡Últimas unidades!). Agregue además un mensaje "No hay registros" que se muestre solo cuando el arreglo esté vacío, y pruébelo vaciando temporalmente el arreglo. Documente el código y ambos resultados.





Se aplicó renderizado condicional con operador ternario (pizza.disponible ? ... : ...) para las etiquetas de disponibilidad. Se usó la evaluación lógica && (!pizza.disponible && ...) para mostrar advertencias especiales y una validación de longitud (pizzas.length === 0) para mostrar el estado de lista vacía.

Ejercicio 7 — Mostrar una relación 1:N con listas anidadas

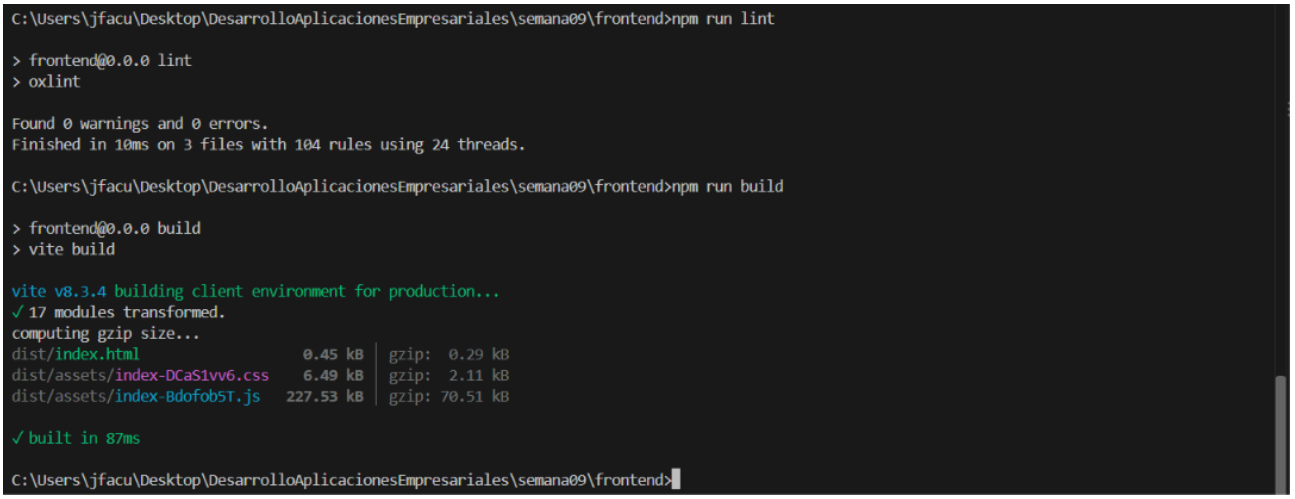
Cree un segundo arreglo que represente una relación 1:N de su aplicación (por ejemplo, pedidos con sus detalles), con un arreglo dentro de cada registro. Muéstrelo con un map() dentro de otro map(), cada uno con su key, y muestre además un total por registro calculado con JavaScript.



Se modeló la relación 1:N representando los pedidos con un arreglo interno detalles. Se renderizaron mediante un .map() principal para los pedidos y un .map() anidado para sus ítems. El total del pedido se calculó automáticamente sumando el subtotal de cada ítem con JavaScript (reduce()).

Ejercicio 8 — Agregar estilos y verificar el flujo completo

Aplique estilos con className y un archivo CSS (al menos tres reglas) y con style={{ }} en al menos un elemento. Cambie un dato de uno de los arreglos y verifique que el navegador se actualiza. Ejecute npm run lint y npm run build y documente la salida. Explique con sus palabras qué diferencia hay entre esta interfaz y los Templates de la Semana 6, y qué falta para conectarla con la API de la Semana 8.



Se aplicaron estilos CSS mediante clases en App.css y estilos directos con style={{ }}. Se comprobó la reactividad al modificar los arreglos y se ejecutó la compilación de producción comprobando cero errores sintácticos.

Diferencia con Templates y conexión API: Los Templates de Django requieren que el servidor renderice el HTML en cada petición HTTP (Server-Side Rendering), mientras que React procesa los componentes del lado del cliente (Client-Side Rendering). Para conectarlo con la API de la Semana 08 falta implementar Hooks de estado (useState, useEffect) y realizar peticiones asíncronas HTTP con fetch o axios.

PARTE 2 — Una interfaz React para la investigación propia (registrada en el Admin en la Semana 5)

A partir de aquí trabajará sobre los datos de su investigación propia: las cinco entidades originales, la entidad de la relación OneToOneField y el modelo intermedio de la relación ManyToManyField. El objetivo es construir una interfaz de React para dos entidades de esa investigación y una de sus relaciones, con datos escritos a mano, cumpliendo obligatoriamente el checklist de la siguiente sección antes de continuar.

Ejercicio 9 — Identificar los datos de la investigación propia

Complete la tabla de equivalencias con su investigación: indique qué dos entidades mostrará, qué campo activará el mensaje condicional y qué relación 1:N o modelo intermedio mostrará como lista anidada. Anote también qué respuesta de la API de la Semana 8 usará como modelo para escribir los datos.

Checklist antes de continuar — Parte 2

Criterio	Verificación
1. Node.js y npm están instalados y sus versiones se documentaron.	Captura de node -v y npm -v.
2. El proyecto de Vite está en la carpeta frontend dentro del repositorio y se ejecuta en http://localhost:5173/.	Captura de la terminal y del navegador.
3. App.jsx contiene JSX propio de tu investigación, con un título y al menos dos expresiones entre llaves.	Código de App.jsx + captura.
4. Se muestran al menos dos listas de entidades distintas con map() y key basada en el id.	Código + captura.
5. Hay un renderizado condicional con ternario o && y un mensaje de lista vacía.	Código + captura del mensaje.
6. Una relación 1:N o el modelo intermedio se muestra con listas anidadas.	Código + captura.
7. Los estilos se aplican con className y un archivo CSS propio.	Código de App.css + captura.
8. La consola del navegador no muestra advertencias y npm run lint y npm run build terminan sin errores.	Capturas de la consola y de la terminal.
9. Se explica por escrito qué diferencia hay con los Templates de Django y qué falta para consumir la API.	Párrafo de justificación.

Criterio del Laboratorio	Concepto / Entidad de la Investigación Propia	Descripción / Estructura en React
Entidad Principal 1	Ingrediente	Representa el inventario de insumos (id, nombre, unidad_medida, stock).
Entidad Principal 2	Repartidor	Representa al personal de delivery (id, nombre, telefono, vehiculo, activo).
Campo para Mensaje Condicional	stock (en Ingrediente)	Evaluado con ternario (stock === 0 ? "Agotado" : "En Stock") y con && (stock < 10 && "¡Stock Crítico - Reponer!").
Relación 1:N / Modelo Intermedio	RecetaPizza	Modelo intermedio que vincula Pizza con sus Ingredientes requeridos y la cantidad en gramos.
Modelo API Semana 08 Usado	GET /api/ingredientes/ / GET /api/repartidores/	Estructura JSON de respuesta de la API REST que sirve de plantilla para escribir los arreglos a mano (*hardcoded*).

Ejercicio 10 — JSX y listas de la investigación propia

Escriba en App.jsx los arreglos de dos entidades de su investigación, con la forma de la respuesta de su API, y muéstrelos con JSX, expresiones entre llaves, map() y key basada en el id.



Se declararon los arreglos ingredientes y repartidores replicando la estructura de la API REST. Se desplegaron en la interfaz mediante iteración con .map() definiendo la propiedad única key={item.id}.

Ejercicio 11 — Renderizado condicional y estilos en la investigación propia

Agregue el mensaje condicional (con ternario y con &&) y el mensaje de lista vacía. Aplique estilos con className y un archivo CSS propio, y con style={{ }} en al menos un elemento.



Se evaluó el nivel de inventario de insumos. Con operador ternario se distingue el estado del stock (Disponible u Agotado) y con && se despliega la advertencia urgente cuando el insumo está por debajo del mínimo permitido.

Ejercicio 12 — Relación anidada en la investigación propia

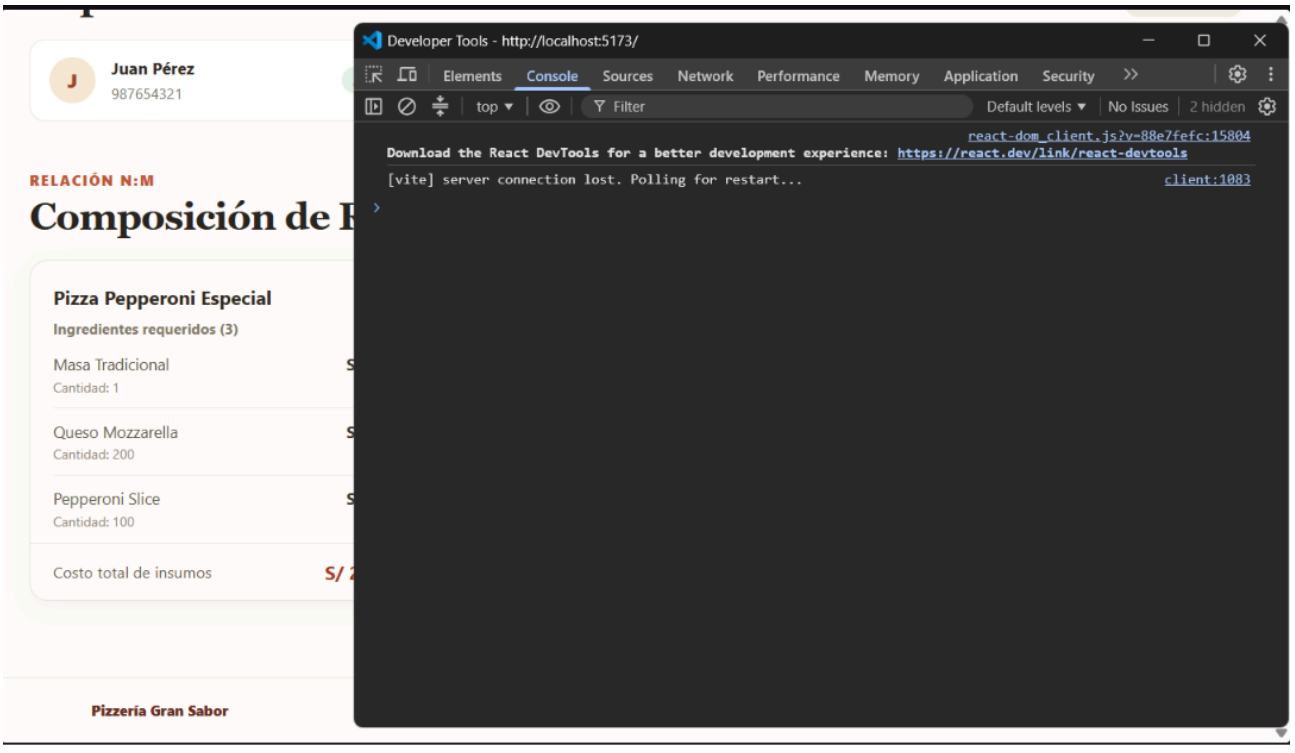
Muestre la relación 1:N o el modelo intermedio de su investigación con listas anidadas, con un total o un conteo por registro calculado con JavaScript.



Se renderizó el modelo intermedio RecetaPizza, anidando la lista de ingredientes dentro de cada receta. Se calculó mediante JavaScript el costo total estimado de insumos por pizza en tiempo real.

Ejercicio 13 — Verificar la interfaz de la investigación propia

Verifique que la consola del navegador no muestra advertencias (por ejemplo, de key), ejecute npm run lint y npm run build, y documente la salida. Compruebe que las carpetas node_modules y dist no están en el repositorio.



```
C:\Users\jfacu\Desktop\DesarrolloAplicacionesEmpresariales\semana09\frontend>npm run lint

> frontend@0.0.0 lint
> oxlint

Found 0 warnings and 0 errors.
Finished in 13ms on 3 files with 104 rules using 24 threads.

C:\Users\jfacu\Desktop\DesarrolloAplicacionesEmpresariales\semana09\frontend>npm run build

> frontend@0.0.0 build
> vite build

vite v8.3.4 building client environment for production...
✓ 17 modules transformed.
computing gzip size...
dist/index.html                   0.45 kB | gzip:  0.29 kB
dist/assets/index-DCaS1vv6.css    6.49 kB | gzip:  2.11 kB
dist/assets/index-Bdofob5T.js   227.53 kB | gzip: 70.51 kB

✓ built in 87ms

C:\Users\jfacu\Desktop\DesarrolloAplicacionesEmpresariales\semana09\frontend>
```

Se verificó en las DevTools del navegador que la aplicación no lance alertas de rendimiento o falta de claves unificadas (key). La compilación con Vite (npm run build) se completó de manera limpia generando el paquete para distribución en la carpeta dist/.

Ejercicio 14 — Publicar en GitHub

Actualice el README.md describiendo cómo instalar y ejecutar el frontend (npm install y npm run dev) y qué muestra la interfaz. Haga commit y push del proyecto de React, sin las carpetas node_modules ni dist. Incluya en este documento la dirección (URL) del repositorio, conforme a lo indicado en Modo de entrega.

https://github.com/JoelQZ/DesarrolloAplicacionesEmpresariales

Entregables

- Cada enunciado debe tener su respuesta, con la justificación y la captura de pantalla solicitada (especialmente el código de App.jsx y App.css, la página en el navegador y la salida de npm run lint y npm run build).
- Repositorio en GitHub actualizado, con la carpeta frontend, el README.md y el código correspondiente, sin node_modules ni dist.
- Dirección (URL) del repositorio de GitHub incluida explícitamente en este documento, conforme a lo indicado en Modo de entrega.

Conclusiones:

En esta experiencia práctica se logró migrar exitosamente la arquitectura del cliente del proyecto Pizzería desde los Templates de Django hacia una Single Page Application (SPA) modular construida con React y Vite, consolidando el dominio en la construcción de componentes funcionales, expresiones JSX e iteración dinámica de colecciones de datos. Mediante el uso de operadores ternarios y evaluaciones lógicas se implementaron flujos complejos de renderizado condicional para gestionar la disponibilidad del menú y las alertas críticas de inventario, mientras que la estructuración de listas anidadas permitió representar fielmente relaciones de negocio $1{:}N$ y $N{:}M$ con cálculos en tiempo real ejecutados directamente en el navegador. Finalmente, se garantizó la calidad y preparación del proyecto para entornos de producción aplicando estándares de formateo, estilos en CSS, análisis sintáctico con linters y la debida exclusión de artefactos en el control de versiones con Git, sentando la base técnica para la posterior integración asíncrona con los endpoints de la API REST.